

LAPORAN PRAKTIKUM PBO

Relasi Antar Objek (Class Diagram & Asosiasi)

Disusun Oleh:

Endy Pardilian 2511531017

Dosen Pengampu:

NURFIAH, S.ST, M.Kom

Asisten Pratikum:

EzzaAddini



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

LangkahPengerjaan:

A. Praktikum

1. Class Rekening

```
package pekan2;
import java.util.ArrayList;

public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;

    ArrayList<Transaksi> riwayatTransaksi;

    public Rekening (String nomor, String nama, double saldoAwal) {
        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;
        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + "
berhasil dibuat dengan saldo Rp" + saldo);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit",
nominal);
            riwayatTransaksi.add(trxBaru);
            System.out.println("Setor tunai Rp" + nominal + "
berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih
dari 10000!");
        }
    }

    public void tarikTunai(double nominal) {
        if (nominal < 10000) {
            System.out.println("Gagal: Minimal tarik 10000!");
        } else if (nominal > saldo) {
            System.out.println("Gagal : Saldo anda tidak
cukup!");
        } else {
            saldo -= nominal;
            String idTrx = "TRX-T-" + System.currentTimeMillis();
            Transaksi trxBaru = new
Transaksi(idTrx, "Debit", nominal);
            riwayatTransaksi.add(trxBaru);
            System.out.println("Tarik tunai Rp" + nominal + "
berhasil. Saldo anda saat ini: " + saldo);
        }
    }
}
```

```

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir : Rp" + saldo);
        System.out.println("-----");
    }
    public void cetakMutasi() {
        System.out.println("--- MUTASI REKENING ---");
        if (riwayatTransaksi.isEmpty()) {
            System.out.println("Belum ada transaksi pada rekening ini");
        } else {
            for (Transaksi trx : riwayatTransaksi) {
                trx.cetakDetail();
            }
        }
        System.out.println("-----");
    }
}

```

Class Rekening pada versi ini masih berfungsi sebagai model/blueprint yang merepresentasikan sebuah rekening bank, namun kini dikembangkan lebih lanjut dengan kemampuan mencatat riwayat transaksi. Setiap rekening tidak hanya menyimpan data identitas dan saldo, tetapi juga menyimpan daftar seluruh transaksi (setor/tarik) yang pernah terjadi, dengan memanfaatkan class lain bernama Transaksi sebagai "pencatat" setiap kejadian transaksi. Ini menunjukkan adanya hubungan antar class (asosiasi/komposisi): satu objek Rekening memiliki banyak objek Transaksi di dalamnya.

a. public Rekening

```

public Rekening (String nomor, String nama, double saldoAwal) {
    nomorRekening = nomor;
    namaPemilik = nama;
    saldo = saldoAwal;
    this.riwayatTransaksi = new ArrayList<>();
    System.out.println("Rekening atas nama " + namaPemilik + "
berhasil dibuat dengan saldo Rp" + saldo);

}

    If (selisih == 1) {
        Return false;
    }

    Return true;
}

```

Constructor ini bertugas menginisialisasi data rekening baru: menyimpan nomor rekening, nama pemilik, dan saldo awal ke atribut masing-masing, sama seperti versi sebelumnya. Yang membedakan adalah baris `this.riwayatTransaksi = new ArrayList` ini membuat `ArrayList` kosong untuk menampung transaksi di masa depan, sehingga setiap rekening baru langsung memiliki "buku catatan" yang siap diisi sejak awal dibuat. Tanpa baris ini, program akan error (`NullPointerException`) saat mencoba menambahkan transaksi nanti karena list-nya belum ada.

b. `public void setorTunai`

```
public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
        String idTrx = "TRX-S-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit",
nominal);
        riwayatTransaksi.add(trxBaru);
        System.out.println("Setor tunai Rp" + nominal + "
berhasil. Saldo saat ini: Rp" + saldo);
    } else {
        System.out.println("Gagal: Nominal setor harus lebih
dari 10000!");
    }
}
```

Method ini menangani proses setor tunai. Berbeda dari versi sebelumnya, validasi di sini hanya mengecek apakah nominal lebih besar dari 0 (bukan lagi minimal 10.000), meski pesan errornya masih menyebut "lebih dari 10000" ini kemungkinan kelalaian penulisan (bug) karena logika kondisi sudah diubah tapi pesannya belum diperbarui. Jika nominal valid, saldo bertambah, lalu dibuat ID transaksi unik menggunakan "TRX-S-" + `System.currentTimeMillis()` (memanfaatkan waktu sistem saat ini dalam milidetik agar ID selalu berbeda). Setelah itu, dibuat objek `Transaksi` baru dengan jenis "Kredit" (karena uang masuk), lalu objek tersebut ditambahkan ke dalam `riwayatTransaksi` agar tercatat. Jika nominal tidak valid, transaksi ditolak tanpa ada pencatatan.

c. `public void tarikTunai`

```
public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Gagal: Minimal tarik 10000!");
    } else if (nominal > saldo) {
        System.out.println("Gagal : Saldo anda tidak
cukup!");
    } else {
        saldo -= nominal;
        String idTrx = "TRX-T-" + System.currentTimeMillis();
        Transaksi trxBaru = new
Transaksi(idTrx, "Debit", nominal);
        riwayatTransaksi.add(trxBaru);
        System.out.println("Tarik tunai Rp" + nominal + "
berhasil. Saldo anda saat ini: " + saldo);
    }
}
```

```
}
```

Method ini menangani penarikan tunai dengan dua validasi seperti sebelumnya: nominal tidak boleh di bawah 10.000, dan tidak boleh melebihi saldo yang tersedia. Jika kedua validasi terpenuhi, saldo dikurangi, lalu dibuat ID transaksi unik dengan format "TRX-T-" + System.currentTimeMillis() (awalan "T" untuk menandakan tarik/debit, berbeda dari "S" pada setor). Objek Transaksi baru dibuat dengan jenis "Debit" (karena uang keluar), kemudian dimasukkan ke dalam riwayatTransaksi agar transaksi ini tercatat dalam riwayat rekening.

d. public void cekInformasi

```
public void cekInformasi() {  
    System.out.println("--- INFO REKENING ---");  
    System.out.println("No. Rekening : " + nomorRekening);  
    System.out.println("Nama Pemilik : " + namaPemilik);  
    System.out.println("Saldo Akhir : Rp" + saldo);  
    System.out.println("-----");  
}
```

Method ini tidak berubah dari versi sebelumnya hanya menampilkan data ringkas rekening (nomor rekening, nama pemilik, saldo akhir) ke layar tanpa mengubah data apa pun, dan tidak berkaitan dengan riwayat transaksi.

e. PublicvoidcetakMutasi

```
public void cetakMutasi() {  
    System.out.println("--- MUTASI REKENING ---");  
    if (riwayatTransaksi.isEmpty()) {  
        System.out.println("Belum ada transaksi pada  
rekening ini");  
    } else {  
        for (Transaksi trx : riwayatTransaksi) {  
            trx.cetakDetail();  
        }  
    }  
    System.out.println("-----");  
}
```

Ini adalah method baru yang berfungsi untuk menampilkan seluruh riwayat transaksi pada rekening tersebut. Method ini pertama-tama mengecek apakah riwayatTransaksi kosong; jika kosong, ditampilkan pesan bahwa belum ada transaksi. Jika tidak kosong, program melakukan perulangan (for-each) ke setiap objek Transaksi di dalam riwayatTransaksi, lalu memanggil method trx.cetakDetail() pada masing-masing objek untuk menampilkan detail transaksi satu per satu (method cetakDetail() sendiri didefinisikan di dalam class Transaksi, bukan di sini). Dengan method ini, pengguna dapat melihat seperti "mutasi rekening" di bank sungguhan daftar history semua transaksi yang pernah terjadi.

2. Class Main

```

package pekan2;

import java.util.Scanner;

public class Main {
    public static void main(String[] endy) {
        Scanner input = new Scanner(System.in);
        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null;
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while(isRunning) {
            System.out.println("\nMenu Utama: ");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
            System.out.println("6. Cetak Mutasi (Riwayat)");
            System.out.println("0. Keluar");
            System.out.println("Pilih menu: ");

            int pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {
                case 1:
                    System.out.println("Masukan No Rekening: ");
                    String no = input.nextLine();
                    boolean akun = false;
                    for (Rekening r : daftarRekening) {
                        if
(r.nomorRekening.equalsIgnoreCase(no)) {
                            akun = true;
                            break;
                        }
                    }
                    if(akun) {
                        System.out.println("Error: Nomor
rekening " + no + "sudah terdaftar!");
                        break;
                    }
                    System.out.println("Masukan Nama Pemilik: ");
                    String nama = input.nextLine();
                    System.out.print("Masukan Saldo Awal: ");
                    double saldo = input.nextDouble();
                    input.nextLine();
                    Rekening rekeningBaru = new Rekening(no, nama,
saldo);

                    daftarRekening.add(rekeningBaru);

                    akunAktif = rekeningBaru;
                    break;

                case 2:

```

```

        if (akunAktif == null) {
            System.out.println("Error: Mohon maaf,
Anda belum memiliki nomor rekening!");
        } else {
            System.out.println("Masukkan nominal
setor: ");

            double setor = input.nextDouble();
            akunAktif.setorTunai(setor);
        }
        break;

    case 3:
        if (akunAktif == null) {
            System.out.println("Error: Mohon maaf,
Anda belum memiliki nomor rekening!");
        } else {
            System.out.println("Masukkan nominal
tarik: ");

            double tarik = input.nextDouble();
            akunAktif.tarikTunai(tarik);
        }
        break;

    case 4:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum
buka rekening!");
        } else {
            akunAktif.cekInformasi();
        }
        break;

    case 5:
        if (daftarRekening.isEmpty()) {
            System.out.println("Error: Belum ada
rekening yang terdaftar!");
        } else {
            System.out.println("Masukkan nomor
rekening yang ingin diaktifkan: ");
            String nomorDicari = input.nextLine();

            Rekening ditemukan = null;
            for (Rekening r : daftarRekening) {
                if
(r.nomorRekening.equalsIgnoreCase(nomorDicari)) {
                    ditemukan = r;
                    break;
                }
            }

            if (ditemukan != null) {
                akunAktif = ditemukan;
                System.out.println("Berhasil
berpindah ke rekening atas nama " + akunAktif.namaPemilik);
            } else {
                System.out.println("Error: Nomor
rekening '" + nomorDicari + "' tidak ditemukan!");
            }
        }
    }
}

```

```

        }
    }
    break;

    case 6:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum
membuka rekening!");
        } else {
            akunAktif.cetakMutasi();
        }
        break;

    case 0:
        isRunning = false;
        System.out.println("Sistem ditutup.
Terimakasih!");
        break;

    default:
        System.out.println("Pilihan tidak valid!");
    }

    }
    input.close();
}
}

```

Class Main ini berfungsi sebagai pengendali program (controller) yang menjalankan seluruh alur interaksi dengan pengguna dalam sistem perbankan mini. Dibandingkan versi sebelumnya, class ini kini mendukung banyak rekening sekaligus (disimpan dalam `ArrayList<Rekening>`), kemampuan berpindah antar akun, serta fitur baru untuk mencetak riwayat/mutasi transaksi. Main sendiri tidak memproses logika transaksi secara langsung, melainkan hanya menampilkan menu, menangkap input pengguna, lalu meneruskan perintah tersebut ke method yang sudah disediakan oleh class Rekening.

a. publicclass Main

```

package pekan2;
import java.util.Scanner;

publicclassMain1{
    publicstaticvoidmain(String[]args){
        Scannerinput=newScanner(System.in);
        RekeningakunAktif=null;
        booleanisRunning=true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while(isRunning){
            System.out.println("\nMenu Utama:");
            System.out.println("1. Buka Rekening Baru");

```



```

        System.out.println("2. Setor Tunai");
        System.out.println("3. Tarik Tunai");
        System.out.println("4. Cek Informasi Rekening");
        System.out.println("0. Keluar");
        System.out.print("Pilih menu: ");

        int pilihan=input.nextInt();
        input.nextLine();

        switch(pilihan){

```

Method ini adalah titik masuk (entrypoint) program. Di awal method, dibuat objek Scanner untuk membaca input dari keyboard, variabel `akunAktif` bertipe `Rekening` yang sengaja diinisialisasi dengan nilai null karena belum ada rekening yang dibuat saat program baru dijalankan, serta variabel `booleanisRunning` yang menjadi saklar agar program terus berjalan dalam perulangan menu sampai pengguna memilih keluar. Perlu diperhatikan bahwa versi ini berbeda dari versi sebelumnya: di sini tidak ada `ArrayList<Rekening>`, sehingga program hanya bisa menangani satu rekening aktif pada satu waktu — setiap kali menu 1 dipilih, rekening lama akan otomatis tertimpa oleh rekening baru.

b. Switch-Case

1. Case 1 Buka Rekening Baru

```

case1:

        System.out.print("Masukkan No Rekening: ");
        Stringno=input.nextLine();
        System.out.print("Masukkan Nama Pemilik: ");
        Stringnama=input.nextLine();
        System.out.print("Masukan Saldo Awal: ");
        doublesaldo=input.nextDouble();

        akunAktif=newRekening(no,nama,saldo);
        break;

```

Bagian ini meminta pengguna memasukkan nomor rekening, nama pemilik, dan saldo awal secara berurutan. Setelah semua data terkumpul, program membuat objek `Rekening` baru dengan memanggil constructor (`new Rekening(no, nama, saldo)`), lalu hasilnya langsung disimpan ke variabel `akunAktif`.

Output:

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 5
Pilihan tidak valid!

```

```

=== SISTEM PERBANKAN MINI ===
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 12345
Masukkan Nama Pemilik: Endy
Masukkan Saldo Awal: 100000
Rekening atas nama Endy berhasil dibuat dengan saldo Rp100000.0

```

2. Case 2 Setor Tunai

```

case2:
    if(akunAktif==null){
        System.out.println("Error: Mohon maaf,
Anda belum memiliki nomor rekening!");
    }else{
        System.out.print("Masukkan nominal
setor: ");
        doublesetor=input.nextDouble();
        akunAktif.setorTunai(setor);
    }
    break;

```

Program terlebih dahulu memeriksa apakah akunAktif masih null. Jika iya, artinya belum ada rekening yang dibuat sehingga transaksi tidak bisa dilakukan dan pesan error ditampilkan. Jika sudah ada rekening aktif, program meminta nominal setor dari pengguna, lalu memanggil methodsetorTunai() milik objek akunAktif — seluruh validasi nominal (minimal setor) diproses di dalam class Rekening, bukan di Main.

Output:

```

=== SISTEM PERBANKAN MINI ===
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 2
Error: Mohon maaf, Anda belum memiliki nomor rekening!

```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 2
Masukkan nominal setor: 10000
Setor tunai Rp10000.0 berhasil. Saldo saat ini: Rp110000.0

```

3. Case 3 Tarik Tunai

```

case3:
    if(akunAktif==null){
        System.out.println("Error: Mohon maaf,
Anda belum memiliki nomor rekening!");
    }else{

```

```

                                System.out.println("Masukkan nominal
tarik: ");
                                double tarik=input.nextDouble();
                                akunAktif.tarikTunai(tarik);
                                }
                                break;

```

Alurnya serupa dengan setor tunai: program memastikan ada akun aktif terlebih dahulu, kemudian meminta nominal tarik dari pengguna dan memanggil method `tarikTunai()` milik `akunAktif`. Semua validasi terkait batas minimal dan kecukupan saldo diproses di dalam class `Rekening`, sehingga class `Main` hanya berperan meneruskan input pengguna ke method yang bersangkutan.

Output:

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Error: Mohon maaf, Anda belum memiliki nomor rekening!

```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik:
9000
Gagal: Minimal tarik 10000!

```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik:
10000
Tarik tunai Rp10000.0 berhasil. Saldo anda saat ini: 10000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik:
11000
Gagal : Saldo anda tidak cukup!

```

4. Case 4 Cek Informasi Rekening

```

case4:
                                if(akunAktif==null){
                                    System.out.println("Error: Anda belum
membuka rekening!");
                                }else{
                                    akunAktif.cekInformasi();
                                }
                                break;

```

Program memeriksa apakah akunAktif masih null terlebih dahulu. Jika belum ada rekening yang dibuka, pesan error ditampilkan. Jika sudah ada, methodcekInformasi() milik akunAktif dipanggil untuk menampilkan nomor rekening, nama pemilik, dan saldo akhir ke layar.

Output:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 4
Error: Anda belum membuka rekening!
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 12345
Nama Pemilik : Endy
Saldo Akhir  : Rp10000.0
-----
```

5. Case 5 GantiAkun

```
case5:
    if(daftarRekening.isEmpty()){
        System.out.println("Error: Belum ada
rekening yang terdaftar!");
    }else{
        System.out.println("Masukkan nomor
rekening yang ingin diaktifkan: ");
        String nomorDicari=input.nextLine();

        Rekening ditemukan=null;
        for(Rekening r:daftarRekening){

            if(r.nomorRekening.equalsIgnoreCase(nomorDicari)){
                ditemukan=r;
                break;
            }
        }

        if(ditemukan!=null){
            akunAktif=ditemukan;
            System.out.println("Berhasil
berpindah ke rekening atas nama "+akunAktif.namaPemilik);
        }else{
            System.out.println("Error: Nomor
rekening '"+nomorDicari+"' tidak ditemukan!");
        }
    }
    break;
```

Bagian ini memungkinkan pengguna berpindah ke rekening lain yang sudah terdaftar. Program pertama-tama memeriksa apakah daftarRekening masih kosong;

jika kosong, pengguna diberi tahu bahwa belum ada rekening yang bisa dipilih. Jika tidak kosong, pengguna diminta memasukkan nomor rekening yang ingin diaktifkan, kemudian program melakukan pencarian di dalam daftarRekening menggunakan perulangan for. Apabila rekening dengan nomor tersebut ditemukan, akunAktif akan diperbarui ke rekening itu disertai pesan konfirmasi; jika tidak ditemukan, pesan error akan ditampilkan kepada pengguna.

Output:

```
=== SISTEM PERBANKAN MINI ===  
  
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
0. Keluar  
Pilih menu:  
5  
Error: Belum ada rekening yang terdaftar!
```

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
0. Keluar  
Pilih menu:  
1  
Masukan No Rekening:  
12345  
Masukan Nama Pemilik:  
Endy  
Masukan Saldo Awal: 100000  
Rekening atas nama Endy berhasil dibuat dengan saldo Rp100000.0
```

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
0. Keluar  
Pilih menu:  
1  
Masukan No Rekening:  
12345  
Error: Nomor rekening 12345sudah terdaftar!
```

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
0. Keluar  
Pilih menu:  
1  
Masukan No Rekening:  
23456  
Masukan Nama Pemilik:  
Pardilian  
Masukan Saldo Awal: 10000  
Rekening atas nama Pardilian berhasil dibuat dengan saldo Rp10000.0
```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
5
Masukkan nomor rekening yang ingin diaktifkan:
12345
Berhasil berpindah ke rekening atas nama Endy

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
5
Masukkan nomor rekening yang ingin diaktifkan:
23456
Berhasil berpindah ke rekening atas nama Pardilian

```

6. Case 0 Keluar

```

case0:
    isRunning=false;
    System.out.println("Sistem ditutup. Terima
    kasih!");
    break;

```

Saat menu ini dipilih, variabel `isRunning` diubah menjadi `false` sehingga perulangan `while` akan berhenti pada pengecekan kondisi berikutnya, disertai pesan penutup sebagai tanda program selesai dijalankan.

Output:

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 0
Sistem ditutup. Terima kasih!

```

3. Class Transaksi

```

package pekan2;

public class Transaksi {
    String idTransaksi;
    String jenis;
    double nominal;

    public Transaksi(String idTrx,String jenis,double nominal) {
        this.idTransaksi = idTrx;
        this.jenis = jenis;
        this.nominal = nominal;
    }
    public void cetakDetail() {

```

```

        System.out.println("ID: " + idTransaksi + " | Jenis: " +
jenis + " | Nominal: Rp" + nominal);
    }
}

```

Class Transaksi merupakan class baru yang berfungsi sebagai model (cetakan data) untuk satu catatan mutasi pada sebuah rekening. Setiap kali terjadi setoran atau tarik tunai yang berhasil, satu objek Transaksi dibuat dan disimpan dalam riwayat rekening. Class ini memiliki tiga atribut, yaitu idTransaksi sebagai penanda unik transaksi, jenis yang berisikategori transaksi ("Kredit" untuk uang masuk dan "Debit" untuk uang keluar), serta nominal yang menyimpan jumlah uang yang ditransaksikan.

a. Constructor Transaksi

Constructor menerima tiga parameter dan menyimpannya ke atribut objek menggunakan kata kunci this. Penggunaan this diperlukan karena nama parameter jenis dan nominal sama dengan nama atributnya, sehingga this.jenis = jenis berarti "isi atribut milik objek ini dengan nilai dari parameter".

b. Public void cetakDetail

Method ini menampilkan satu baris rincian transaksi ke layar dengan format ID, jenis, dan nominal. Method ini hanya membaca data dan tidak mengubah apa pun. Dengan menempatkan method pencetak di dalam class Transaksi itu sendiri, objek Transaksi mampu "menjelaskan dirinya sendiri" sehingga class lain cukup memanggil cetakDetail() tanpa perlu mengetahui isi atributnya.

B. Tugas Praktikum

Tugas 1 :

Integrasi Rekam Jejak Penarikan (30%)

- Buka kembali method tarikTunai(double nominal) di dalam file Rekening.java.
- Tambahkan logika perekaman objek Transaksi seperti pada contoh setorTunai. Pastikan jenis transaksinya dicatat sebagai "Debit" dan kodenya menggunakan prefiks "TRX-T-". Transaksi hanya boleh dicatat jika penarikan berhasil (saldo mencukupi).publicstaticbooleanisValidPin

public void tarikTunai

```
public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Gagal: Minimal tarik 10000!");
    }else if(nominal > saldo){
        System.out.println(" Gagal : Saldo anda tidak
cukup!");
    }else {
        saldo -= nominal;
        String idTrx = "TRX-T-" +System.currentTimeMillis();
        Transaksi trxBaru = new
Transaksi(idTrx,"Debit",nominal);
        riwayatTransaksi.add(trxBaru);
        System.out.println("Tarik tunai Rp" + nominal + "
berhasil. Saldo anda saat ini: " + saldo);
    }
}
```

Method ini menangani penarikan tunai dengan dua validasi seperti sebelumnya: nominal tidak boleh di bawah 10.000, dan tidak boleh melebihi saldo yang tersedia. Jika kedua validasi terpenuhi, saldo dikurangi, lalu dibuat ID transaksi unik dengan format "TRX-T-" + System.currentTimeMillis() (awalan "T" untuk menandakan tarik/debit, berbeda dari "S" pada setor). Objek Transaksi baru dibuat dengan jenis "Debit" (karena uang keluar), kemudian dimasukkan ke dalam riwayatTransaksi agar transaksi ini tercatat dalam riwayat rekening.

Tugas 2:

Fitur Cetak Mutasi Rekening (70%)

- Buat sebuah method baru di dalam class Rekening.java dengan nama cetakMutasi().
- Di dalam method tersebut, gunakan struktur perulangan for-each untuk menelusuri seluruh data di dalam koleksi riwayatTransaksi. Panggil

method cetakDetail() milik setiap objek Transaksi. Jika riwayat kosong, cetak pesan: "Belum ada transaksi pada rekening ini".

- Buka file Main.java, tambahkan menu baru: "6. Cetak Mutasi (Riwayat)". Panggil method cetakMutasi() milik akunAktif saat pengguna memilih menu tersebut

Case 6 Cetak Mutasi

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum
membuka rekening!");
    } else {
        akunAktif.cetakMutasi();
    }
    break;
```

Program memeriksa apakah ada akun aktif terlebih dahulu; jika tidak ada, pesan error ditampilkan. Jika ada, program memanggil akunAktif.cetakMutasi(), yang akan menampilkan seluruh riwayat transaksi (baik setor maupun tarik) yang pernah dilakukan oleh rekening tersebut, dengan memanfaatkan data yang tersimpan di riwayatTransaksi pada objek Rekening.

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu:
6
--- MUTASI REKENING ---
ID: TRX-S-1791289624690 | Jenis: Kredit | Nominal: Rp100000.0
ID: TRX-T-1791289632747 | Jenis: Debit | Nominal: Rp90000.0
-----
```

C. Link Repositori GitHub

https://github.com/endypardilian/PrakPBO_C_2511531017/tree/pekan2/PrakPBO_C_2511531017/src/pekan2